

BACKEND NOTES

How a real FastAPI project is organised

Folders • Routes • requirements.txt • How to run

*A simple Product CRUD API using FastAPI, PostgreSQL, and cursor SQL***One idea for the whole document**

Do not put everything in one main.py. Split the work: URL, JSON check, SQL, database connection. Companies do this so many people can work on the same project without breaking each other.

1. Why folders like a real company project

In a first program, all code is often in one file. That is fine for 50 lines. A company API has many URLs, many tables, and many people. If everything is in one file, it becomes hard to find a bug and hard to add a new feature.

The common rule is: **one job per folder**.

Layer	Folder	Job	Company reason
App start	main.py	Create FastAPI + CORS + attach routes	One place to start the server
Connection	app/database.py	Open Postgres	DB settings stay in one file
Schema	app/schemas/	JSON shape (name, price, quantity)	Invalid JSON is rejected early
Service	app/services/	SQL with cursor	Business work is not mixed with URLs
Controller	app/controllers/	HTTP URLs	Easy to see all APIs
Libraries	requirements.txt	What pip must install	Same versions on every machine

This is the same idea used in Java (controller / service / repository), in Spring Boot, in Node (routes / services), and in Python FastAPI. The names change a little. The idea does not.

2. Project folders

```

postgres-db-demo/
■■■ main.py                start the app
■■■ requirements.txt       libraries to install
■■■ app/
    ■■■ database.py        Postgres connection
    ■■■ schemas/
        ■■■ product.py     JSON body
        ■■■ services/
            ■■■ product_service.py  cursor SQL (CRUD)
            ■■■ controllers/
                ■■■ product_controller.py  GET POST PUT DELETE URLs

```

Empty `__init__.py` files make a folder a Python package, so other files can write from `app.services.product_service import ProductService`.

2.1 main.py — start the application

This file does three things only: create the FastAPI app, allow the React UI (CORS), and attach the product routes. It does not contain SQL.

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.controllers import product_router

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(product_router)

```

CORS: the UI is `http://localhost:5173` and the API is `http://localhost:8000`. Different ports mean different origins. The browser blocks the call unless the API says the UI is allowed. Swagger can work without CORS. The React UI cannot.

2.2 app/database.py — open PostgreSQL

Companies keep connection details in one place. If the password or database name changes, only this file is edited. `row_factory=dict_row` makes each row a dictionary like `{"id": 1, "name": "Mouse"}` so the UI can read field names.

```

def get_connection():
    return psycopg.connect(
        host="localhost",
        dbname="postgres",
        user="postgres",
        password="postgres123",
        port=5432,
        row_factory=dict_row,
    )

```

2.3 app/schemas/ — JSON contract

A schema is not the database table. It is the JSON that the client sends. If price is text, Pydantic rejects the request before SQL runs. That is why companies separate schema from SQL.

```
class Product(BaseModel):
    name: str
    price: float
    quantity: int
```

2.4 app/services/ — SQL / business logic

The service talks to Postgres with a cursor. This is the same simple logic as a single-file demo: connect, execute, fetch, commit, close. The only change is: this code lives in a service class, so the controller stays small.

```
conn = get_connection()
cursor = conn.cursor()
cursor.execute("SELECT * FROM products")
products = cursor.fetchall()
cursor.close()
conn.close()
return products
```

Service method	SQL idea
get_all_products	SELECT * FROM products
create_product	INSERT ... RETURNING *
update_product	UPDATE ... WHERE id = ? RETURNING *
delete_product	DELETE FROM products WHERE id = ?

2.5 app/controllers/ — HTTP routes

A controller maps a URL + method to a service method. It should not contain SQL. If the URL changes, only this file changes. If the SQL changes, only the service changes. That is how teams work in parallel.

```
@router.get("/products")
def get_products(service: ProductService = Depends(get_product_service)):
    return service.get_all_products()
```

Depends means FastAPI creates the service for the request. This is called dependency injection. The function asks for a ProductService. It does not build it by hand.

3. API routes

CRUD means Create, Read, Update, Delete. HTTP already has words for that.

Action	HTTP	URL	Controller function	Service method
Read all	GET	/products	get_products	get_all_products
Create	POST	/products	create_product	create_product
Update one	PUT	/products/{product_id}	update_product	update_product
Delete one	DELETE	/products/{product_id}	delete_product	delete_product

Sample JSON for POST and PUT:

```
{
  "name": "Wireless Mouse",
  "price": 799,
  "quantity": 25
}
```

How one POST request travels:

```
Browser / UI
  POST http://localhost:8000/products
    |
    v
Controller  receives URL and JSON
    |
    v
Schema      checks name, price, quantity
    |
    v
Service     INSERT with cursor, commit
    |
    v
PostgreSQL  new row, id is generated
    |
    v
JSON back to the UI
```

4. requirements.txt — how companies install libraries

Python projects do not assume every machine already has FastAPI. Companies list libraries in requirements.txt. Anyone can recreate the same environment with pip.

```
fastapi      # write APIs
uvicorn      # run the server
psycopgg[binary] # talk to PostgreSQL
pydantic     # validate JSON
```

Why not install packages by hand each time? Because one person may have FastAPI 0.100 and another 0.115. A shared requirements file keeps the team aligned. In larger companies this file is also used in CI servers and Docker images so production matches the laptop.

Create a virtual environment first, so project libraries do not mix with system Python:

```
cd postgres-db-demo
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

4.1 Why these two commands: venv and activate

Those two commands create a **private Python** for this project, then **switch to it**.

1. python3 -m venv .venv

Creates a folder named `.venv` with its own Python and pip. Without this, `pip install fastapi` goes into the system Python. Then one project may need FastAPI 0.100 and another 0.115. Packages mix and break. `.venv` is this project's own toolbox.

2. source .venv/bin/activate

Turns that toolbox on in the current terminal. After activate, `python` and `pip` mean this project's Python, not the global one. The prompt usually shows `(.venv)`.

Windows:

```
.venv\Scripts\activate

Laptop Python          <- do not dump project libraries here
|
+-- .venv              <- only for postgres-db-demo
    fastapi
    uvicorn
    psycpg
```

One sentence

`venv` creates a clean Python for this app. `activate` tells the terminal to use it. Then `pip install -r requirements.txt` installs libraries into `.venv` only. After the terminal is closed, run `activate` again before `uvicorn`. To leave it: `deactivate`.

Create `.venv` only once per project. Activate it every time a new terminal is opened.

5. How to run the app

Before start

- PostgreSQL is running
- A table named `products` exists (`id`, `name`, `price`, `quantity`)
- `app/database.py` has the correct username, password, and database name

Start the API

```
cd postgres-db-demo
source .venv/bin/activate
uvicorn main:app --reload
```

`main:app` means: file `main.py`, variable `app`. `--reload` restarts the server when a Python file is saved.

What	URL
API home / docs	http://localhost:8000/docs
List products	http://localhost:8000/products
React UI (if used)	http://localhost:5173

Test in Swagger in this order: GET /products, POST /products, PUT /products/{id}, DELETE /products/{id}.

6. If something fails

Problem	Likely cause	Fix
ModuleNotFoundError: app	Wrong folder	cd into postgres-db-demo, then run uvicorn
password authentication failed	Wrong DB password	Edit app/database.py
relation products does not exist	Table not created	CREATE TABLE in pgAdmin
UI cannot load products	CORS or API not running	Start uvicorn, keep CORS for port 5173
npm / node not found	Node.js missing (UI only)	Install Node.js LTS from nodejs.org
pip installs into Anaconda / system	Forgot activate	source .venv/bin/activate, then pip install

7. Four lines to remember

```

Controller = URL
Schema     = JSON
Service    = SQL (cursor)
database.py = Postgres connection

python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
uvicorn main:app --reload
http://localhost:8000/docs

```

What companies add later (not required in this demo)

Alembic migrations, .env for passwords, tests, Docker, login/auth, logging. The folder idea stays the same when those are added.